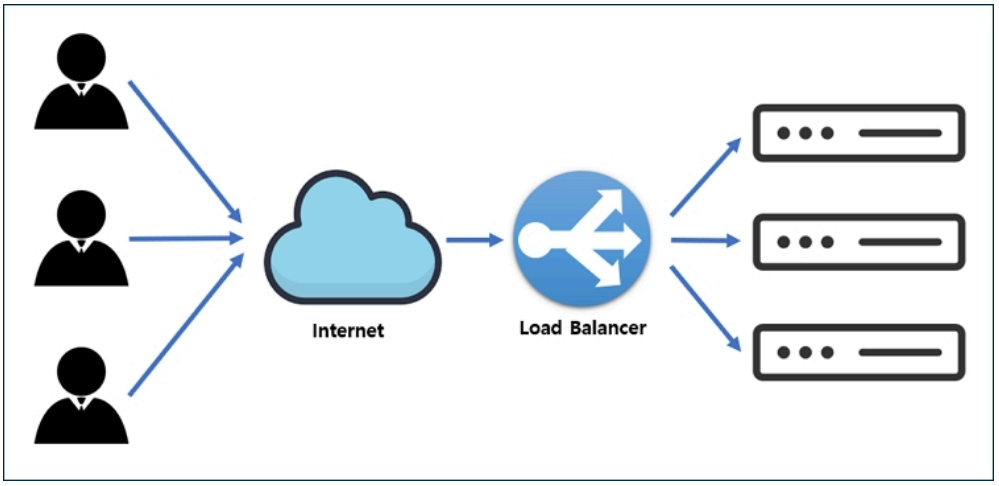
8/13 5회차 스터디



▼ 예습해오기!

안녕하세요! 5주차 스터디 자료 깃허브에 올려두었습니다!
환경세팅 파일만 먼저 확인해주시고, 오후 중으로 이론 파일 수정해서 올려두겠습니다.
시간 되시면 한 번씩 읽어와주시면 좋을 것 같아요.

이번 주차는 PetClinic 앱을 두 개로 늘려서, Nginx가 요청을 두 앱에 나눠 보내도록 만듭니다. 앱 하나가 멈춰도 서비스가 살아있는지 확인해보는 게 목표일 것 같습니다.

영상은 아래 세 개 참고해주세요!

1. https://youtu.be/42Q65H8ch7U?si=A_6aHA453yS2P3CP
   18분짜리 영상으로 조금ㅎㅎ 긴데, 이번 주차에 저희가 할 내용을 전반적으로 다루는 영상입니다. 앱을 flask로 사용하는 부분은 조금 달라서 시간 여유 충분하시면 보셔도 좋을 것 같아요.

2. https://youtu.be/ilnUBOVeBCc?si=3hblhUTNyJdnpry3
   저번주에 지태님이 보내주셨던 영상이랑 비슷한데, NginX가 하는 역할에 대해 조금 더 자세히 설명해주는 영상입니다.

3. https://youtu.be/7FpSPSlJj-0?si=vZfVn76b9Q1KwvN_
   그리고 저는 정말 배경지식이 없어서 봤던 영상인데, Nginx가 웹서버,리버스 프록시,로드밸런서 역할을 어떻게 다 겸하는지 한 편에 정리해주는 영상입니다. 4주차에 했던 것과 5주차에 할 게 사실 같은 프로그램의 다른 기능이라는 게 이 영상 보면 좀 잡히더라구요.....!

책도 관련된 부분 있어서 같이 남겨둡니다. <이것이 리눅스다>에서

- 4.1.10 마운트와 CD/DVD/USB의 활용 (190~206p)

- 4.4 네트워크 관련 설정과 명령어(273p) ~ 4.7 서비스와 소켓(298p)

요 부분들이 5주차 내용이랑 관련된 부분인데, 다 읽어오실 필요는 없고 훑어만 보셔도 충분할 것 같아요! 그럼 더위 조심하시고, 목요일 2시에 뵙겠습니다 :)

😳 5회차 스터디

https://github.com/likelion-server-study/WEEK5

▼ 6주차 미리보기

6주차 — Prometheus·Grafana로 장애를 발견하고 복구 확인하기

이전 주차에서는 같은 PetClinic 앱을 `app1`, `app2` 두 컨테이너로 실행하고, Nginx가 요청을 분산하도록 구성했습니다. 앱 하나가 멈추더라도 Nginx가 다른 앱으로 요청을 보내 서비스를 유지하는 과정도 확인했습니다.

이번 주차에서는 장애를 화면과 로그로만 확인하지 않고, 실제 운영에서 사용하는 모니터링·부하 테스트 도구를 이용해 서버 상태를 수치와 그래프로 관찰합니다.

```
사용자 요청
→ Nginx
→ app1 / app2
→ PostgreSQL

k6
→ 일정한 시험 요청 발생

Prometheus
→ app1·app2의 상태와 성능 지표 수집

Grafana
→ Prometheus가 수집한 지표를 그래프와 경고로 표시
```

이번 주에 사용하는 도구

도구	역할
Spring Boot Actuator·Micrometer	PetClinic의 요청 수, 응답시간, JVM, DB 연결 상태 등을 측정
Prometheus	app1·app2의 지표를 주기적으로 수집하고 시간 순서대로 저장
Grafana	Prometheus에 저장된 지표를 대시보드와 경고로 표시
k6	일정한 요청을 보내 응답시간과 실패율 측정

도구	역할
Docker Restart Policy	비정상 종료된 app 컨테이너를 자동으로 다시 실행
Nginx	app 하나가 실패하면 정상 app으로 요청을 우회

Prometheus는 CNCF의 Graduated 프로젝트이며, Spring Boot도 Micrometer를 이용한 Prometheus 연동을 공식 지원합니다. k6는 개발자·QA·SRE가 부하 및 신뢰성 시험에 사용하는 오픈소스 도구입니다.

6주차에 확인할 문제

서버가 정상인지 계속 로그를 직접 확인해야 하는가

→ PetClinic에 Spring Boot Actuator와 Prometheus용 Micrometer를 추가합니다.

→ Prometheus가 다음 주소에서 app1·app2의 지표를 각각 수집합니다.

```
app1:8080/actuator/prometheus
app2:8080/actuator/prometheus
```

Prometheus는 Nginx를 거치지 않고 Compose 내부에서 각 app에 직접 접근합니다. 그래야 두 앱의 지표가 섞이지 않습니다.

서버 상태를 시간에 따라 비교할 수 있는가

→ Grafana를 Prometheus와 연결해 다음 항목을 대시보드로 확인합니다.

- app1·app2 실행 상태
- 서버별 요청 처리량
- HTTP 응답시간
- HTTP 오류율
- JVM 메모리 사용량
- JVM 스레드 수
- DB 커넥션 풀 사용량

Grafana는 직접 서버 상태를 측정하지 않습니다. PetClinic이 지표를 만들고, Prometheus가 수집·저장하며, Grafana는 저장된 지표를 조회해 보여 줍니다.

일정한 조건으로 서버 성능을 비교할 수 있는가

→ k6로 캐시되지 않는 동적 경로인 `/owners`에 일정한 요청을 보냅니다.

정상 상태에서 다음 값을 먼저 기록합니다.

- 전체 요청 수
- 초당 요청 수
- 평균 응답시간
- 상위 95% 응답시간
- HTTP 요청 실패율
- app1·app2가 각각 처리한 요청 수

이 값을 장애 발생 전의 기준으로 사용합니다.

app 하나가 멈추면 어떤 변화가 생기는가

→ k6 요청을 계속 보내면서 app1의 Java 프로세스를 비정상 종료합니다.

```
app1 종료
→ Prometheus에서 app1 상태가 DOWN으로 변경
→ Nginx가 app2로 요청 우회
→ app2 요청 처리량 증가
→ 사용자의 요청이 계속 성공하는지 확인
```

Nginx 로그에서는 어느 app에 연결을 시도하고 재시도했는지 확인합니다. Grafana에서는 app1의 중단과 app2의 부하 증가를 확인합니다.

Prometheus가 app1의 장애를 감지하더라도 Nginx의 요청 분배를 직접 제어하지는 않습니다.

- Nginx: 실제 요청 실패를 보고 다른 app으로 우회
- Prometheus: app 상태와 성능 지표 수집
- Grafana: 수집된 지표를 시각화하고 경고 표시

종료된 app을 자동으로 복구할 수 있는가

→ app1과 app2에 Docker Restart Policy를 적용합니다.

app1의 주 프로세스가 비정상 종료되면 Docker가 컨테이너를 다시 실행하는지 확인합니다.

```
app1 비정상 종료
→ Docker가 app1 재실행
→ PetClinic 기동 완료
→ Prometheus에서 app1 상태 정상화
→ Nginx가 app1을 다시 사용
→ app1·app2 요청 분산 복구
```

Docker Healthcheck와 Restart Policy는 역할이 다릅니다.

- Healthcheck: 컨테이너가 정상인지 판정
- Restart Policy: 컨테이너의 주 프로세스가 종료됐을 때 다시 실행

`docker compose stop app1`처럼 사용자가 정상적으로 중지한 컨테이너는 자동 재시작 시험에 사용하지 않습니다.

장애를 사람이 계속 보고 있지 않아도 알 수 있는가

→ Grafana에 app 상태를 확인하는 간단한 경고 규칙을 만듭니다.

```
app1 정상 → Normal
app1 중지 → Alerting
app1 복구 → Normal
```

이번 실습에서는 실제 메신저나 이메일 발송보다, 장애 발생과 복구에 따라 경고 상태가 바뀌는 것을 확인합니다.

실습 진행 순서

1. 5주차의 `app1`, `app2`, Nginx, PostgreSQL 구성을 확인합니다.
2. PetClinic에 Prometheus용 Micrometer 의존성을 추가합니다.
3. PetClinic JAR와 Docker 이미지를 다시 만듭니다.
4. Prometheus와 Grafana를 Compose 서비스로 추가합니다.
5. Prometheus가 app1·app2의 지표를 각각 수집하는지 확인합니다.
6. Grafana에서 요청량·응답시간·JVM·DB 지표를 확인합니다.
7. k6로 `/owners`에 일정한 요청을 보내 정상 상태를 측정합니다.
8. app1을 비정상 종료하고 Nginx의 우회와 지표 변화를 확인합니다.
9. Docker가 app1을 자동으로 다시 실행하는지 확인합니다.

10. app1 복구 후 두 앱이 다시 요청을 나누는지 확인합니다.
11. Grafana 경고 상태가 장애와 복구에 맞게 변하는지 확인합니다.

장애 전·중·후 비교

확인 항목	정상 상태	app1 장애 중	app1 복구 후
app1 상태	UP	DOWN	UP
app2 상태	UP	UP	UP
app2 요청량	정상 분산	증가	정상 분산
사용자 응답	200	200 유지 여부 확인	200
HTTP 실패율	기준값	변화 확인	기준값과 비교
상위 95% 응답시간	기준값	변화 확인	기준값과 비교
Nginx 로그	app1·app2 분산	app1 실패 후 app2 우회	다시 분산
Grafana 경고	Normal	Alerting	Normal

성공 기준

- Prometheus가 app1·app2의 지표를 각각 수집한다.
- Grafana에서 요청량·응답시간·JVM·DB 상태를 확인할 수 있다.
- k6로 같은 조건의 시험을 반복할 수 있다.
- app1 장애가 Prometheus와 Grafana에 나타난다.
- app1 장애 중에도 Nginx가 app2로 요청을 전달한다.
- Docker가 비정상 종료된 app1을 다시 실행한다.
- app1 복구 후 두 앱이 다시 요청을 나누어 처리한다.
- Grafana 경고가 장애 발생과 복구 상태에 맞게 변한다.
- 장애 전·중·후의 응답시간과 실패율을 비교해 기록한다.

참고 자료

- Prometheus 공식 문서
- CNCF Prometheus 프로젝트
- Grafana 공식 문서
- Grafana k6 공식 문서

- Spring Boot Metrics 공식 문서
- Docker Restart Policy 공식 문서